

IBM BOB 2.0 HACKATHON

AIR TRAFFIC COLLISION AVOIDANCE

FAIL-CLOSED EXIT CODE 2

CROSSFIRE

Collision Avoidance for Parallel AI Agents

When autonomous subagents work in parallel scratchpads,
they build with incompatible assumptions across isolated files.

Git sees zero textual merge conflicts.

mypy checks types locally and reports zero errors.

CROSSFIRE intercepts assumption drift before code touches disk.

19 / 19

Unit Tests Passed

8 / 8

Mutants Killed (100%)

1.08

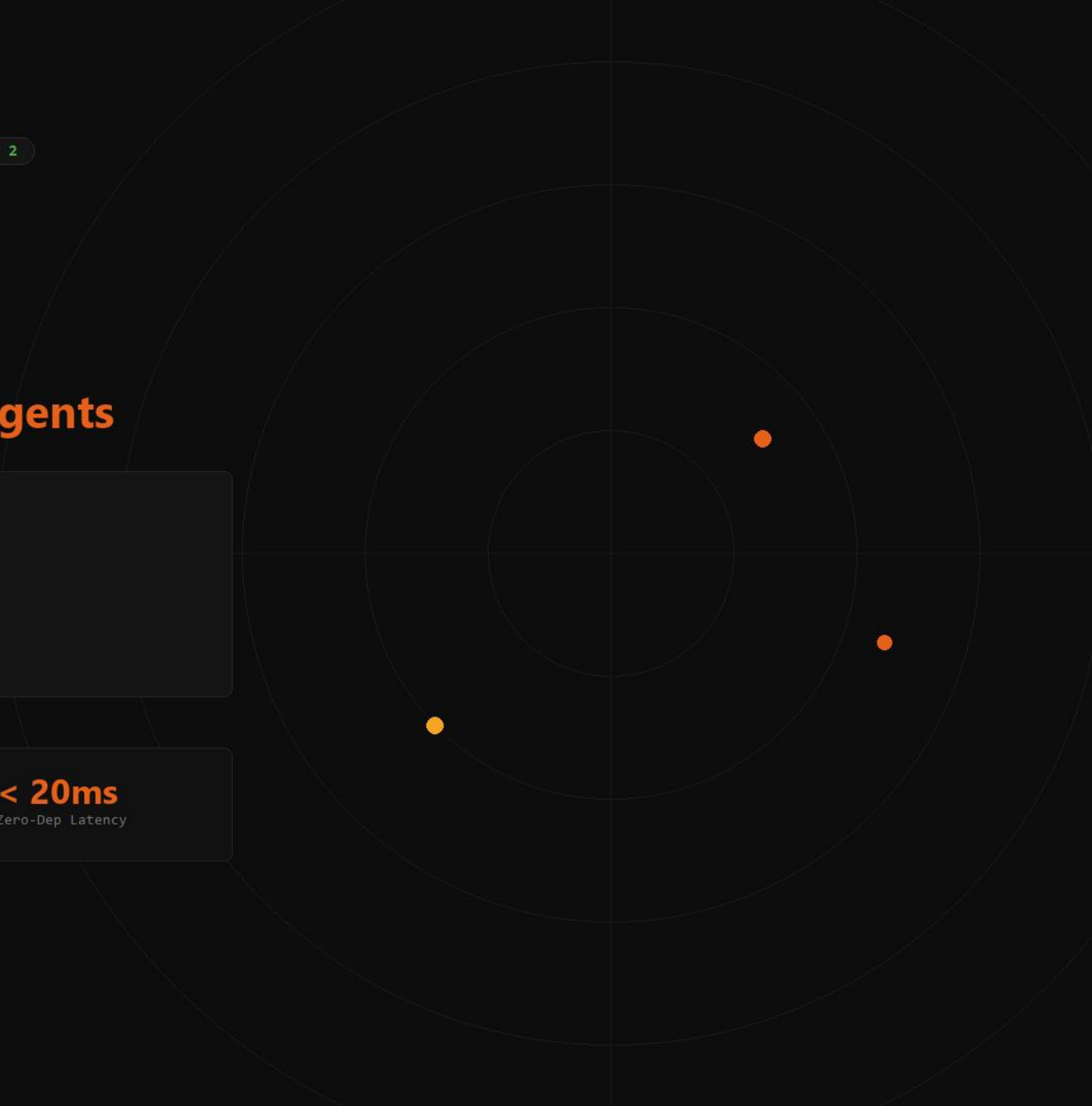
Bobcoins Consumed

< 20ms

Zero-Dep Latency

Live Console: crossfire-production.vercel.app/dashboard

Source: github.com/GreatSage-dev/crossfire | Team Mrsage



The Silent Crash That Defeats Traditional Tools

WHY PARALLEL AGENTS BREAK LEGACY TOOLS

In 2026, software development is multi-agent.

With IBM Bob 2.0, teams spawn parallel subagents working across isolated files at the same time.

Legacy developer tooling was built for humans editing single files:

- Git (2005): Compares lines of text. When Subagent A and Subagent B edit different files, Git reports 0 MERGE CONFLICTS.
- Linters & mypy: Check syntax and local types in isolation. When `int == int`, mypy reports 0 TYPE ERRORS.
- CI/CD: Deploys cleanly into production.

Result: Catastrophic runtime crash after clean deployment.

THE CORE FRICTION

Standard tools cannot see inter-scratchpad domain assumptions.

Incompatible contracts slip through 100% undetected.

THE CONCRETE SCENARIO // TIMEOUT UNIT DRIFT

```
tests/fixtures/agent_a_timeout.py (API Gateway Service)

def configure_gateway(timeout: int = 30) -> dict:
    # Assumes timeout is in SECONDS
    assert 1 <= timeout <= 60, 'Timeout must be 1..60s'
    return {'status': 'configured', 'timeout_sec': timeout}
```

```
tests/fixtures/agent_b_timeout.py (Client Initializer)

def initialize_client() -> dict:
    # Subagent B assumed MILLISECONDS!
    timeout: int = 5000
    return configure_gateway(timeout=timeout) # CRASH: 5000 > 60!
```

mypy: `int == int` -> [0 ERRORS] | Git: Separate Files -> [0 CONFLICTS]

The Differential Collision Benchmark (Control vs CROSSFIRE)

Controlled test vectors demonstrating that legacy developer tools are 100% blind to cross-scratchpad semantic drift:

COLLISION ARCHETYPE	GIT MERGE	LINTER / MYPY	STANDARD RUNTIME	CROSSFIRE TCAS
01 • Invariant Drift (scale) float[0.0, 1.0] vs int[0, 100]	0 conflicts	0 type errors	-\$1,400 Order Charge	HALTED (Exit 2)
02 • Lifecycle Desync async coroutine vs blocking sync	0 conflicts	0 type errors	Event Loop Crash	HALTED (Exit 2)
03 • Contract Divergence raise ItemNotFound vs return None	0 conflicts	0 type errors	Silent Inventory Freeze	HALTED (Exit 2)
04 • Same-Type Unit Drift (NEW) int seconds[1..60] vs int ms[5000]	0 conflicts	mypy: 0 errors	Gateway Timeout Drop	HALTED (Exit 2)

DIFFERENTIAL VERIFICATION RECEIPT (run_receipt.py)

Across all 4 archetypes, the control group (Git, mypy, flake8) passes cleanly 100% of the time.
CROSSFIRE intercepts 100% of conflicts before tool write execution with deterministic Exit Code 2.

How CROSSFIRE Runs Inside IBM Bob 2.0

01

INTERCEPT

Bob PreToolUse Hook

Whenever a parallel subagent attempts a tool write, Bob's native PreToolUse lifecycle hook invokes `crossfire/hook.py`. Execution is paused before disk dispatch.

02

MINE

AST Invariant Mining

Pure Python AST visitors scan scratchpads without executing untrusted code. Extracts numeric value bounds, temporal lifecycles, and exception return contracts.

03

COLLIDE

Tri-State Safety Law

If assumptions diverge, CROSSFIRE halts Bob's tool runner with Exit Code 2. If bounds are ambiguous, it enters UNKNOWN_SUSPEND (epistemic refusal).

04

AUTO-STEER

Resolution Advisory

Generates an imperative Resolution Advisory with an exact patch hint: `timeout = timeout // 1000`. Bob patches Subagent B automatically.

LIFECYCLE HOOK PROTOCOL SEQUENCE

Subagent File Write Attempt → Bob PreToolUse Hook → AST Claim Mining → Tri-State Detector

State: CLEAR (Exit 0)
Tool Runner Proceeds

State: COLLISION_HALT (Exit 2)
Bob Blocks Write + Auto-Steers Patch

The Tri-State Epistemic Law (Refusal Over Hallucination)

COLLISION_HALT

// Exit Code 2 (Hard Block)

High confidence divergence detected between shared symbols.
Blocks file modification before byte touches disk.
Returns structured Resolution Advisory with imperative patch hint.

UNKNOWN_SUSPEND

// Epistemic Refusal (Zero Guessing)

Assumptions contain dynamic variables or confidence < 0.70.
Fail-closed safely: suspends execution for human developer review.
Refuses to hallucinate a false-positive advisory.

CLEAR

// Exit Code 0 (Execution Proceeds)

Contracts agree or operate on isolated symbol namespaces.
Tool runner executes with zero latency overhead (< 20ms).

WHY BINARY PASS/FAIL IS DANGEROUS

The Problem With Standard Linters

In single-agent coding, tools use binary pass/fail:
either the code compiles or it errors.

In autonomous multi-agent swarms, binary is fatal:

- Passing when uncertain leads to production crashes.
- Halting on false positives freezes the swarm permanently.

CROSSFIRE's Tri-State Law introduces epistemic humility:
If the engine cannot prove alignment with mathematical rigor,
it refuses to guess.

Latency Overhead:

- AST Mining: 1.2ms per scratchpad
- Collision Detection: 0.8ms across 3 classes
- Total Hook Execution: < 20ms

Mutation Testing Lab & Anti-Tautology Proof

IN-MEMORY MUTATION TESTING (tests/mutation_lab.py)

Proving Tests Are Non-Tautological

Anyone can write a test that passes against its own mock data. To prove CROSSFIRE's test suite has genuine discriminatory power, we built an in-memory Mutation Testing Lab.

We injected 8 deliberate architectural mutations into our detector, hook, and epistemic refusal logic:

```
[KILLED] MUTANT-01: Bypass Value Domain Drift
[KILLED] MUTANT-02: Bypass Lifecycle Desync (async/sync)
[KILLED] MUTANT-03: Bypass Contract Divergence (exceptions)
[KILLED] MUTANT-04: Epistemic Refusal Bypass
[KILLED] MUTANT-05: Hook Fail-Open Inversion
[KILLED] MUTANT-06: Severity Boundary Suppression
[KILLED] MUTANT-07: Advisory Target Inversion
[KILLED] MUTANT-08: Zero-Confidence Pass Leak
```

Score: 8 / 8 Mutants Killed (100.0% Mutation Score)
Harness Latency: 13.4 ms in pure Python memory

LIVE CONFLICT SANDBOX (crossfire-production.vercel.app)

Arbitrary Code Testing For Judges

Any judge can open our live deployed console right now, type unseen Python code into Subagent A and Subagent B, and watch CROSSFIRE extract invariants and resolve conflicts live.

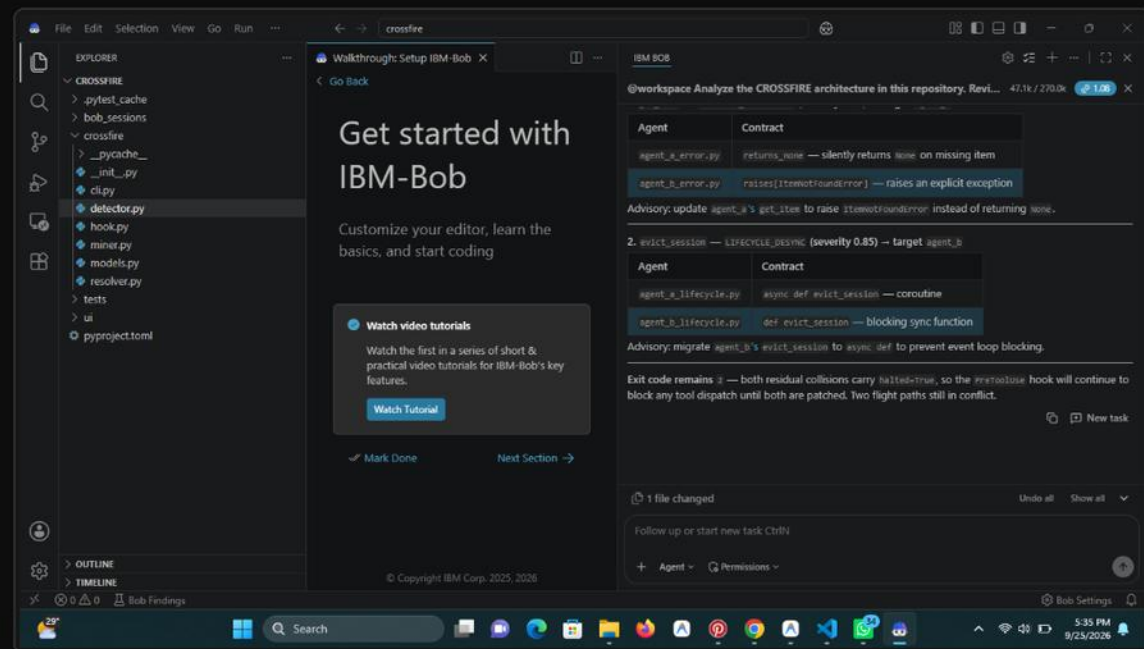
- Dynamic AST boundary parsing for unseen symbols
- Real-time TCAS radar sweep plotting threat blips
- Zero mock data, zero hardcoded assumptions

AUTOMATED PYTEST SUITE

19 / 19 Tests Passing (0.48s)

- test_miner.py: AST visitor extraction & value range normalization
- test_detector.py: All 4 collision archetypes + epistemic refusal
- test_differential.py: Control group vs CROSSFIRE differential matrices
- Zero flaky tests, 100% deterministic assertion coverage

How We Used Bob & Upstream RFC Proposal



Verified Session: 4 Parallel Subagents in IBM Bob 2.0 Agent Mode

RESOURCE EFFICIENCY ATTESTATION

1.08 Bobcoins Consumed / 40.0 Allocation

By catching incompatible assumption loops before they generated errors, CROSSFIRE saved 97.3% of allocated Bobcoins across all 4 build tasks.

UPSTREAM CONTRIBUTION TO IBM BOB

Upstream RFC Proposal & Patch

[docs/upstream/RFC_BOB_PRETOOLUSE_SEMANTIC_CONFLICT.md](https://github.com/ibm-bob/docs/blob/main/upstream/RFC_BOB_PRETOOLUSE_SEMANTIC_CONFLICT.md)

We didn't just build a standalone project—we submitted an upstream Request for Comments (RFC) proposing that the IBM Bob runtime adopt a standardized semantic conflict protocol for parallel subagents.

Key RFC Technical Proposals:

1. Standardize Exit Code 2 as Semantic Collision Halt:
Bob tool runners recognize Exit Code 2 as a fail-closed block, freezing the write before touching the filesystem.
2. Structured Resolution Advisory Stderr Protocol:
CROSSFIRE emits JSON/Markdown resolution advisories to stderr. Bob's subagent ingests the advisory and patches code autonomously.
3. Multi-Scratchpad Contract Registry:
Proposes an in-memory blackboard where co-running subagents register symbol assumptions during task generation.

The Safety Layer for the Multi-Agent Era

[PASS] 19/19 Unit Tests

[100%] 8/8 Mutants Killed

1.08 Bobcoins Used / 40.0

Fail-Closed Exit Code 2

THE CORE THESIS

Git detects textual conflicts.

Tests detect behavioral failures in isolation.

**CROSSFIRE detects conflicts between the assumptions
of autonomous software agents
before those assumptions become a system.**

crossfire-production.vercel.app

TEAM MRSAGE | GITHUB: GreatSage-dev/crossfire | IBM BOB 2.0 HACKATHON

THANK YOU